Introduction to Java and Object-Oriented Programming

Building a Foundation for OOP

Agenda

- **Part 1:** What is Java? (The Basics)
- **Part 2:** Quick Revision of Programming Fundamentals
  - Variables, Data Types, Control Flow
  - Arrays and Methods
- **Part 3:** The Big Shift: Introduction to Classes & Objects
- **Part 4:** Diving Deeper into OOP Concepts
  - Classes vs. Objects
  - Attributes and Methods
  - The static and final keywords
- **Part 5:** Procedural Programming vs. Object-Oriented Programming

Introduction to Java

- **What is Java?**
  - A high-level, object-oriented programming language.
  - Known for its principle: **"Write Once, Run Anywhere" (WORA)** .
- **Key Features:**
  - **Platform Independent:** Java code is compiled into bytecode, which runs on the Java Virtual Machine (JVM). This makes it portable across different operating systems.
  - **Object-Oriented:** Everything in Java (except primitives) is an object, promoting modular and reusable code.
  - **Robust & Secure:** It has strong memory management and no explicit pointers, making it secure.
- **Common Uses:** Android development, enterprise web applications, big data technologies, and more.

Variables and Data Types

- **Variable:** A container that holds data. It has a name, a type, and a value.
- **Data Types:** Defines the type of data a variable can store.
  - **Primitive Types:** Store simple values.
    - `int (integer): int age = 25;`
    - `double (decimal): double price = 19.99;`
    - `boolean (true/false): boolean isJavaFun = true;`
    - `char (single character): char grade = 'A';`
  - **Non-Primitive Types:** Store references to objects.
    - `String, Arrays, Classes, etc.`

Revision: Operators and Control Structures

- **Operators:** Symbols that perform operations on variables and values.
  - **Arithmetic: +, -, *, /, %**
  - **Relational: ==, !=, >, <, >=, <=**
  - **Logical: && (AND), || (OR), ! (NOT)**
- **Control Structures:** Determine the flow of execution.
  - **if else**
  - **swich case**

Control Structures

if else

```
if (temperature > 30) {
  System.out.println("It's hot!");
} else {
  System.out.println("It's pleasant.");
}
```

switch case

```
switch case (value){
  case 0:
    System.out.println(“OFF");
  case 1:
    System.out.println(“ON");
}
```

Loops: for loop

```
// Count from 1 to 5
for (int i = 1; i <= 5; i++) {
    System.out.println("Iteration " + i);
 }

// Countdown example
for (int i = 3; i >= 1; i--) {
    System.out.print(i + " ");
 }

// Iterating through an array
String[] fruits = {"Apple", "Banana", "Cherry"};
for (int i = 0; i < fruits.length; i++) {
 System.out.println("Fruit " + i + ": " + fruits[i]);
 }
```

Loops: while

```
int number = 1;
while (number <= 100) {
    System.out.println("Current number: " +
   number);
    number *= 2;
}


int userChoice = 0;
int counter = 0;
do {
   counter++;
   System.out.println("Menu option " + counter + "
   selected");
} while (userChoice != 1);
```

Loops

for Loops

```
for (int i = 0; i < 5; i++) {
  System.out.println("Count: " + i);
}
```

While loop

```
if (temperature > 30) {
  System.out.println("It's hot!");
} else {
  System.out.println("It's pleasant.");
}
```

Revision: Arrays

- **What is an Array?** A container object that holds a fixed number of values of a **single data type**.
- **Key Points:**
  - Index starts at 0.
  - Length is fixed once created.

Arrays: Syntax and Example

```
// Declaration and Creation
int[] numbers = new int[5]; // An array of 5
integers

// Initialization
numbers[0] = 10;
numbers[1] = 20;

// Alternate way (Declaration, Creation &
Initialization)
String[] names = {"Alice", "Bob", "Charlie"};

// Accessing elements
System.out.println(names[1]); // Output: Bob
```

Revision: Methods

- **What is a Method?** A block of code that performs a specific task. It's used to define the behavior of an object or a class.
- **Why use methods?**
  - **Code Reusability:** Write once, use many times.
  - **Modularity:** Breaks a complex problem into smaller parts.
- **Method Structure (Signature):**

```
returnType methodName(parameters) {
    // method body
    return value; // if returnType is not 'void'
}
```

Methods: Example

```
public static int add(int a, int b) {
    int sum = a + b;
    return sum;
}
// Call the method: int result = add(5, 3);
```

Part 3: Introduction to Classes and Objects

- **So far, we've used procedural building blocks (methods, variables).**
- **Object-Oriented Programming (OOP)** is a different way to structure our code.
- It models real-world things as **objects**.
- An **object** has:
  - **State (Attributes/Properties):** What it *has* (e.g., a car has a color, a model, a speed).
  - **Behavior (Methods):** What it *does* (e.g., a car can start, brake, accelerate).
- A **Class** is the blueprint or template from which individual objects are created.

The Blueprint and the House (Analogy)

- **Class = The Blueprint**
  - It defines what attributes a house *will have* (e.g., number of rooms, color).
  - It defines what actions a house *can do* (e.g., open door, turn on light).
  - It exists only on paper (in code).

- **Object = The Actual House**
  - It is a concrete instance built from the blueprint.
  - It has actual values for the attributes (e.g., 3 rooms, blue color).
  - You can live in it (interact with it).
- You can create many different houses (objects) from the same blueprint (class).

Classes and Objects in Java

Defining a Class (The Blueprint):

```
public class Car {
 // Attributes (variables)
 String color;
 String model;
 int year;

 // Methods (behavior)
 void startEngine() {
  System.out.println("The " + model + " engine is starting!");
 }
}
```

Classes and Objects in Java

Creating an Object (The Instance):

```
// 'new' keyword creates the object in memory
Car myFirstCar = new Car();


// Accessing attributes and methods using the
'dot' operator
myFirstCar.color = "Red";
myFirstCar.model = "Tesla Model 3";
myFirstCar.startEngine();
```

Attributes and Methods (A Deeper Look)

- **Attributes (Instance Variables):**
  - Declared inside a class but outside any method.
  - Each object (instance) of the class gets its *own copy* of these variables.
  - They define the state of the object.
- **Methods (Instance Methods):**
  - They define the operations or behaviors that an object can perform.
  - They often operate on the attributes of the object.
  - Example in our Car class:

Attributes and Methods (A Deeper Look)

Example in our Car class:

```
public class Car {
 // Attributes (variables)
 String color;
 String model;
 int year;

 // Methods (behavior)
 void startEngine() {
  System.out.println("The " + model + " engine is starting!");
 }

void displayInfo() {
    System.out.println("Model: " + model + ", Color: " + color
+ ", Year: " + year);
 }

}
```

Constructor

- **Definition:** A special method that is **automatically called** when an object of a class is created (instantiated).
- **Purpose:** To **initialize the new object's state** (set initial values for attributes).
- **Key Characteristics:**
  - It has the **same name** as the class.
  - It does **not** have a return type (not even void).
  - It is called using the new keyword.
- **Syntax:**

```
public class ClassName {
    public ClassName() {
        // Constructor code - runs when an object is made
    }
}
```

Types of Constructors

- **Default Constructor:**
  - If you do **not** define any constructor in a class, Java provides one for free.
  - It takes no parameters and sets numeric attributes to 0, booleans to false, and object references to null.
  - *Note:* Once you create your own constructor, the default one is no longer provided.
- **Parameterized Constructor:**
  - A constructor that **accepts parameters**.
  - Allows you to initialize an object with **specific, custom values** at the time of creation.
  - This is much more flexible than creating an empty object and setting fields line-by-line.

Constructors in Action (Example)

```
public class Car {
    String color;
    String model;
    // --- Parameterized Constructor ---
    public Car(String carColor, String carModel) {
        color = carColor;
        model = carModel;
        System.out.println("A new " + color + " " + model + " has been
   built!");
    }
    void startEngine() {
        System.out.println("The " + model + " is starting...");
    }
}
```

```
// Passing values directly when creating the object
Car car1 = new Car("Red", "Ferrari");
Car car2 = new Car("Blue", "Porsche");

car1.startEngine(); // Output: The Ferrari is starting...
```

The static Keyword

- The static keyword means the member (variable or method) **belongs to the class itself, not to any specific object instance**.
- **Static Variable (Class Variable):** There is only *one* copy of this variable, shared by all objects of the class.
- **Static Method (Class Method):** Can be called without creating an object. It can only directly access other static members.

The static Keyword

Example

```
public class MathUtility {
    // static variable
     public static final double PI = 3.14159;

    // static method
     public static int add(int a, int b) {
         return a + b;
     }
}

// Usage:
double circleArea = MathUtility.PI * radius * radius;
// Called on the class, not an object
int sum = MathUtility.add(5, 3);
```

The final Keyword

- The final keyword is used to create constants or to restrict further modification.
- It can be applied to:
  - **Variables:** The variable's value cannot be changed once assigned (it becomes a constant).

```
final int MAX_SPEED = 120;
// MAX_SPEED = 100; // This would cause a compiler error!
```

  - **Methods:** The method cannot be overridden by a subclass.
  - **Classes:** The class cannot be subclassed (extended).
- **Note:** static final is a common combination to create true class-level constants (e.g., Math.PI)

Procedural vs. Object-Oriented Programming

Feature	Procedural Programming	Object-Oriented Programming
Focus	**Functions / Procedures** that operate on data.	**Objects** that contain both data and functions.
Data	Data is often global or passed between	functions.Data is **encapsulated** (hidden) within objects.
Approach	Top-down approach (break problem into functions).	Bottom-up approach (design objects and their interactions).
Code Reuse	Limited. Functions can be reused.	Extensive. Through classes, objects, and inheritance.
Example	C is a procedural language.	Java, C++, Python are OOP languages.
Analogy	A series of cooking steps (functions) using ingredients (data).	A kitchen with specialized chefs (objects) who have their own ingredients and tools.

Summary and Key Takeaways

- **Java** is a powerful, platform-independent, object-oriented language.
- **Basic Constructs** (variables, loops, arrays, methods) are the building blocks of any Java program.
- **A Class** is a blueprint; **an Object** is an instance of that blueprint.
- Objects have **Attributes** (what they know) and **Methods** (what they do).
- **static** members belong to the class itself.
- **final** creates unchangeable values or restricts inheritance/overriding.
- **OOP** differs from **Procedural Programming** by combining data and behavior into objects, leading to more modular, maintainable, and reusable code.
- **Next Topic:** Constructors, this keyword, and deeper dive into Encapsulation!